

跨平台开发 Taro

刘军 liujun

目录

content



1 自定义组件

2 跨端兼容实现

3 Redux状态管理

4 卷皮项目实战

组件及生命周期

■ 在 Taro 中，除了应用和页面组件有生命周期之外，**Taro 的组件也是生命周期**，如下图所示：

■ 下面我们来编写一个HYButton组件。

□ 创建组件

□ 定义属性

□ 样式编写

□ 定义插槽

□ 定义生命周期

□ 组件可编写页面生命周期吗？

✓ class组件默认不行，需要单独处理

✓ 但是函数组件是支持的

□ 页面可以编写组件生命周期吗？可以

`"classnames": "^2.3.2",`

`"proptypes": "^1.1.0",`

class 组件	Hooks 组件
constructor	useState
getDerivedStateFromProps	useState 里面 update 函数
shouldComponentUpdate	useMemo
render	函数本身
componentDidMount	useEffect
componentDidUpdate	useEffect
componentWillUnmount	useEffect 里面返回的函数
componentDidCatch	无
getDerivedStateFromError	无

跨端兼容方案

■ Taro 的设计初衷就是为了统一跨平台的开发方式，并且已经尽力通过运行时框架、组件、API 去抹平多端差异，但是由于不同的平台之间还是存在一些无法消除的差异，所以为了更好的实现跨平台开发，Taro 中提供了如下的解决方案。

■ 方案一：内置环境变量

- Taro 在编译时提供了一些内置的环境变量来帮助用户做一些特殊处理。
- 通过这个变量来区分不同环境，从而使用不同的逻辑。在编译阶段，会移除不属于当前端的代码，只保留当前端的代码。
- 内置环境变量虽然可以解决大部分跨端的问题，但是会让代码中存在很多逻辑判断的代码，影响了代码的可维护性，而且也让代码变得丑陋。
- 为了解决这种问题，Taro 提供了另外一种跨端开发的方式作为补充。

■ 方案二：统一接口的多端文件

- 开发者可以通过将文件修改成 原文件名 + 端类型 的命名形式（端类型对应着 process.env.TARO_ENV 的取值），不同端的文件代码对外保持统一接口，而引用的时候仍然是 import 原文件名的文件。
- Taro 在编译时，会根据当前编译平台类型，精准加载对应端类型的文件，从而达到不同的端加载其对应端的文件。

内置环境变量

■ 内置环境变量（`process.env.TARO_ENV`），该环境变量可直接使用

□ `process.env.TARO_ENV`，用于判断当前的编译平台类型，有效值为：`weapp` / `swan` / `alipay` / `tt` / `qq` / `jd` / `h5` / `rn`。

□ 通过这个变量来区分不同环境，从而使用不同的逻辑。

□ 在编译阶段，会移除不属于当前平台的代码，只保留当前平台的代码，例如：

```
/** 源码 */
if (process.env.TARO_ENV === 'weapp') {
  require('path/to/weapp/name')
} else if (process.env.TARO_ENV === 'h5') {
  require('path/to/h5/name')
}
```

```
/** 源码 (React JSX) */
<View>
  {process.env.TARO_ENV === 'weapp' && <ScrollViewWeapp />}
  {process.env.TARO_ENV === 'h5' && <ScrollViewH5 />}
</View>
```

■ 注意事项：不要解构 `process.env` 来获取环境变量，请直接以完整书写的方式（`process.env.TARO_ENV`）来进行使用。

统一接口的多端文件

■ **统一接口的多端文件**这一跨平台兼容写法有如下三个使用要点：

- 1.不同端的对应文件一定要**统一接口和调用方式**。
- 2.引用文件的时候，**只需写默认文件名，不用带文件后缀**。
- 3.**最好有一个平台无关的默认文件**，这样在使用 TS 的时候也不会出现报错。

■ 常见有以下使用场景：

□ **多端组件**（属性，方法，事件等需统一）

✓ 针对不同的端写不同的组件代码

□ **多端脚本逻辑**（属性、方法等需统一）

✓ 针对不同的端写不同的脚本逻辑代码

— test.js	Test 组件默认的形式，编译到微信小程序、百度小程序和 H5 之外的端使用的版本
— test.weapp.js	Test 组件的微信小程序版本
— test.swan.js	Test 组件的百度小程序版本
— test.h5.js	Test 组件的 H5 版本

```
import Test from '../..components/test'  
  
<Test argA={1} argA={2} />
```

认识Redux Toolkit (RTK)

■ Redux Toolkit 是官方推荐的编写 Redux 逻辑的方法。

- 以前我们在使用redux的时候，通常会将redux代码拆分在多个文件中，比如：constants、action、reducer 等
- 这种代码组织方式过于繁琐和麻烦，导致代码量过多，也不利于后期管理
- Redux Toolkit 就是为了解决这种编码方式而诞生。
- 并且以前的 createStore 方式已标为过时，而 Redux Toolkit 已成为官方推荐；

■ 安装Redux Toolkit:

```
npm install @reduxjs/toolkit react-redux
```

■ Redux Toolkit的核心API主要是如下几个:

- **configureStore**: 包装createStore以提供简化的配置选项和良好的默认值。
 - ✓ 可自动组合 slice reducer
 - ✓ 可添加其它 Redux 中间件，redux-thunk默认包含，
 - ✓ 默认启用 Redux DevTools Extension
- **createSlice**: 接受切片名称、初始状态值和reducer函数的对象，并自动生成切片reducer，并带有相应的actions。
- **createAsyncThunk**: 接受一个动作类型字符串和一个返回承诺的函数，并生成一个pending/fulfilled/rejected基于该承诺分派动作类型的thunk。简单理解就是专门用来创建异步Action。

创建counter模块的reducer

■ 我们先创建counter模块的reducer：通过createSlice创建一个slice。

■ createSlice主要包含如下几个参数：

□ name：用来标记slice的名词

✓ redux-devtool中会显示对应的名词；

□ initialState：第一次初始化时的值；

□ reducers：相当于之前的reducer函数

✓ 对象类型，并且可以添加很多的函数；

✓ 函数类似于redux原来reducer中的一个case语句；

✓ 函数的参数：

➤ 参数一：state

➤ 参数二：action

□ createSlice 返回值是一个对象

✓ 对象包含所有的 actions 和 reducer；

```
import { createSlice } from '@reduxjs/toolkit'

const counterSlice = createSlice({
  name: "counter",
  initialState: {
    counter: 0
  },
  reducers: {
    addNumber(state, { payload }) {
      state.counter += payload
    },
    subNumber(state, { payload }) {
      state.counter -= payload
    }
  }
})

export const { addNumber, subNumber } = counterSlice.actions

export default counterSlice.reducer
```


store的创建

■ `configureStore`用于创建store对象，常见参数如下：

- ❑ `reducer`，将slice中的reducer可以组成一个对象传入此处；
- ❑ `middleware`：可以使用参数，传入其他的中间件（自行了解）；
- ❑ `devTools`：是否配置devTools工具，默认为true；

```
import { configureStore } from "@reduxjs/toolkit"
import counterReducer from "../counter"
import homeReducer from "../home"

const store = configureStore({
  reducer: {
    counter: counterReducer,
    home: homeReducer
  }
})

export default store
```

■ 在app.js中将store接入应用:

- **Provider**, 内容提供者, 给所有的子或孙子组件提供store对象;
- **store**: 使用configureStore创建的store对象;

```
import { Provider } from 'react-redux'
import store from './store'
class App extends Component {
  render () {
    return (
      <Provider store={store}>
        {this.props.children}
      </Provider>
    )
  }
}
export default App
```

开始使用store

■ 在函数式组件中可以使用 react-redux 提供的 Hooks API 连接、操作 store。

□ **useSelector** 允许你使用 selector 函数从 store 中获取数据 (root state) 。

□ **useDispatch** 返回 redux store 的 dispatch 引用。你可以使用它来 dispatch actions。

□ **useStore** 返回一个 store 引用, 和 Provider 组件引用完全一致。

```
import { Component } from 'react'
import { useSelector } from 'react-redux'

export const CounterComponent = () => {
  const counter = useSelector(state => state.counter)
  return <View>{counter}</View>
}
```

```
import { Component } from 'react'
import { useStore } from 'react-redux'

export const CounterComponent = ({ value }) => {
  const store = useStore()
  return <div>{store.getState()}</div>
}
```

```
import { Component } from 'react'
import { useDispatch } from 'react-redux'

export const CounterComponent = ({ value }) => {
  const dispatch = useDispatch()

  return (
    <View>
      <Text>{value}</Text>
      <Button onClick={() => dispatch({ type: 'increment-counter' })}>
        Increment counter
      </Button>
    </View>
  )
}
```

Redux Toolkit异步Action操作

- 在之前的开发中，我们通过redux-thunk中间件让dispatch中可以进行异步操作。
- Redux Toolkit默认已经给我们继承了Thunk相关的功能：createAsyncThunk

```
export const getHomeMultidataAction = createAsyncThunk("home/multidata", async (payload, extraInfo) => {  
  const res = await axios.get("http://123.207.32.32:8000/home/multidata")  
  return res.data.data  
})
```

- 当createAsyncThunk创建出来的action被dispatch时，会存在三种状态：

- ❑ **pending**：action被发出，但是还没有最终的结果；
- ❑ **fulfilled**：获取到最终的结果（有返回值的结果）；
- ❑ **rejected**：执行过程中有错误或者抛出了异常；

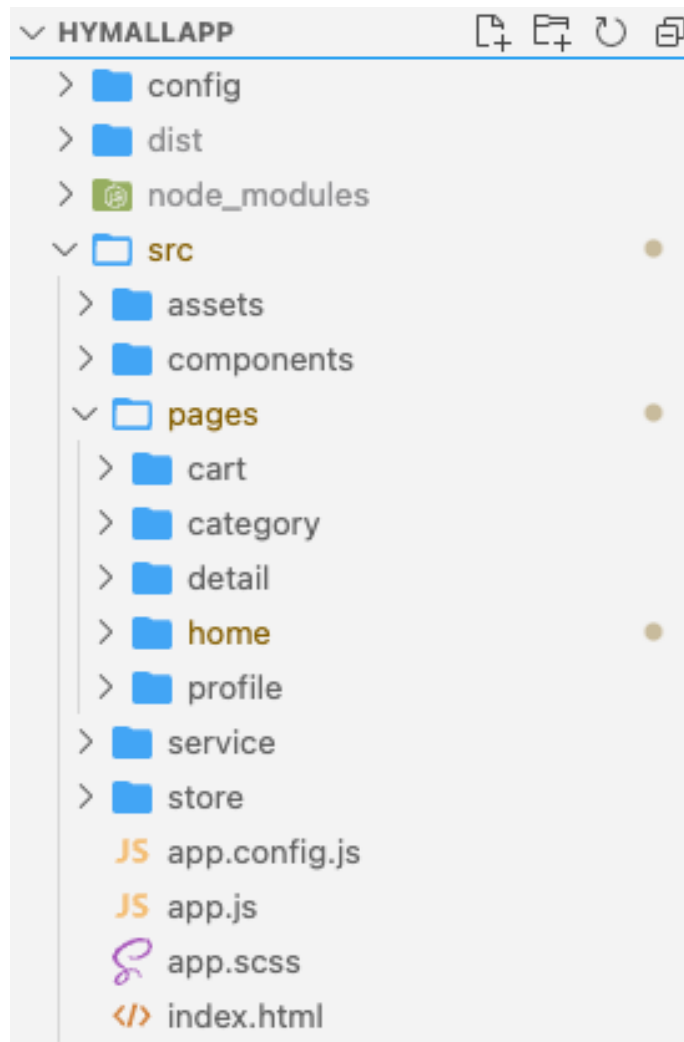
- 我们可以在createSlice的extraReducer中监听这些结果：见右图

```
extraReducers: {  
  [getHomeMultidataAction.pending](state, action) {  
    console.log("getHomeMultidataAction pending", action)  
  },  
  [getHomeMultidataAction.fulfilled](state, { payload }) {  
    console.log("getHomeMultidataAction fulfilled", payload)  
    state.banners = payload.banner.list  
    state.recommends = payload.recommend.list  
  },  
  [getHomeMultidataAction.rejected](state, action) {  
  
  }  
}
```

■ 小程序appid: wxbc30134b589795b0

■ 项目安装的依赖:

- ❑ @reduxjs/toolkit
- ❑ react-redux
- ❑ classnames
- ❑ proptypes



项目打包和部署

■ 多端同步调试

- 可以在 dist 目录下创建一个与编译的目标平台名同名的目录，并将结果放在这个目录下。
- 例如：编译到微信小程序，最终结果是在 `dist/weapp` 目录下；H5打包结果放在 `dist/h5` 目录下
- 好处是，各个平台使用独立的目录互不影响，从而达到多端同步调试的目的，在 `config/index.js` 配置如下：

/config/index.js

```
outputRoot: `dist/${process.env.TARO_ENV}`
```

■ 浏览器端

- 打包： `npm run build:h5`

■ 微信小程序

- 打包： `npm run build:weapp`

- 微信开发者工具打开weapp目录进行预览或发包

对于微信小程序，还需要将 `project.config.json` 文件的 `miniprogramRoot` 的值改为 `dist/weapp/`，这样就可以正常在开发者工具中看到小程序了。